

Machine Learning B (2025)

Home Assignment 6

Yasin Baysal, cmv882

Contents

1	Analysing AdaBoost	2
2	Landcover Classification	5
3	Majority Vote	9
4	Occam's kl-razor vs. PAC-Bayes-kl	13

1 Analysing AdaBoost

Let a training set $T = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\} \subset \mathbb{R}^d \times \{-1, +1\}$ of labelled instances.

Response to Question 1. By induction over b , we have to prove that the weight updates

$$w_i^{(b+1)} = \frac{w_i^{(b)} \exp(-\alpha_b y_i h_b(\mathbf{x}_i))}{\sum_{j=1}^n w_j^{(b)} \exp(-\alpha_b y_j h_b(\mathbf{x}_j))}$$

introduced in AdaBoost can be written as

$$w_i^{(b+1)} = \frac{\exp(-y_i f_b(\mathbf{x}_i))}{\sum_{j=1}^n \exp(-y_j f_b(\mathbf{x}_j))},$$

where h_b is the weak learner fitted in boosting round b , α_b the importance of h_b , and $f_b = \sum_{p \leq b} \alpha_p h_p$.

Proof. We give a proof by induction over b . For every integer $b \geq 0$ and every $i \in \{1, \dots, n\}$,

$$P(b): \quad w_i^{(b+1)} = \frac{\exp(-y_i f_b(\mathbf{x}_i))}{\sum_{j=1}^n \exp(-y_j f_b(\mathbf{x}_j))},$$

where $f_b(\mathbf{x}) = \sum_{p \leq b} \alpha_p h_p(\mathbf{x})$ and by convention $f_0(\mathbf{x}) = 0$. For the base case with $b = 0$, $f_0(\mathbf{x}) = 0$, such that $\exp(-y_i f_0(\mathbf{x}_i)) = \exp(-y_i \cdot 0) = \exp(0) = 1$ and $\sum_{j=1}^n \exp(-y_j f_0(\mathbf{x}_j)) = \sum_{j=1}^n 1 = n$. We know that the round-1 weights in AdaBoost are initialized to $w_i^{(1)} = \frac{1}{n}$.

Therefore, we get that

$$\frac{\exp(-y_i f_0(\mathbf{x}_i))}{\sum_{j=1}^n \exp(-y_j f_0(\mathbf{x}_j))} = \frac{1}{n} = w_i^{(1)},$$

which is exactly $w_i^{(0+1)} = w_i^{(1)}$. Thus, the formula holds at $b = 0$, i.e., the base case $P(0)$ holds. For the inductive step, fix an integer $b \geq 0$ and assume $P(b)$ holds, that is, for all i ,

$$w_i^{(b+1)} = \frac{\exp(-y_i f_b(\mathbf{x}_i))}{\sum_{j=1}^n \exp(-y_j f_b(\mathbf{x}_j))},$$

which is inductive hypothesis. We want to prove that $P(b+1)$, namely

$$w_i^{((b+1)+1)} = \frac{\exp(-y_i f_{b+1}(\mathbf{x}_i))}{\sum_{j=1}^n \exp(-y_j f_{b+1}(\mathbf{x}_j))} = \frac{\exp(-y_i [f_b(\mathbf{x}_i) + \alpha_{b+1} h_{b+1}(\mathbf{x}_i)])}{\sum_{j=1}^n \exp(-y_j [f_b(\mathbf{x}_j) + \alpha_{b+1} h_{b+1}(\mathbf{x}_j)])}.$$

But by AdaBoost's definition of the new weight, we know that

$$w_i^{(b+2)} = w_i^{((b+1)+1)} = \frac{w_i^{(b+1)} \exp(-\alpha_{b+1} y_i h_{b+1}(\mathbf{x}_i))}{\sum_{j=1}^n w_j^{(b+1)} \exp(-\alpha_{b+1} y_j h_{b+1}(\mathbf{x}_j))}.$$

and by substituting the inductive hypothesis into numerator and denominator, then

$$\begin{aligned} w_i^{(b+2)} &= \frac{w_i^{(b+1)} \exp(-\alpha_{b+1} y_i h_{b+1}(\mathbf{x}_i))}{\sum_{j=1}^n w_j^{(b+1)} \exp(-\alpha_{b+1} y_j h_{b+1}(\mathbf{x}_j))} \\ &= \frac{\left(\frac{\exp(-y_i f_b(\mathbf{x}_i))}{\sum_{k=1}^n \exp(-y_k f_b(\mathbf{x}_k))} \right) \exp(-\alpha_{b+1} y_i h_{b+1}(\mathbf{x}_i))}{\sum_{j=1}^n \left(\frac{\exp(-y_j f_b(\mathbf{x}_j))}{\sum_{k=1}^n \exp(-y_k f_b(\mathbf{x}_k))} \right) \exp(-\alpha_{b+1} y_j h_{b+1}(\mathbf{x}_j))} \end{aligned}$$

and when we simplify this expression for $w_i^{(b+2)}$, we obtain that

$$\begin{aligned} w_i^{(b+2)} &= \frac{\left(\frac{\exp(-y_i f_b(\mathbf{x}_i))}{\sum_{k=1}^n \exp(-y_k f_b(\mathbf{x}_k))} \right) \exp(-\alpha_{b+1} y_i h_{b+1}(\mathbf{x}_i))}{\sum_{j=1}^n \left(\frac{\exp(-y_j f_b(\mathbf{x}_j))}{\sum_{k=1}^n \exp(-y_k f_b(\mathbf{x}_k))} \right) \exp(-\alpha_{b+1} y_j h_{b+1}(\mathbf{x}_j))} \\ &= \frac{\frac{1}{\sum_{k=1}^n \exp(-y_k f_b(\mathbf{x}_k))} \exp(-y_i f_b(\mathbf{x}_i)) \exp(-\alpha_{b+1} y_i h_{b+1}(\mathbf{x}_i))}{\frac{1}{\sum_{k=1}^n \exp(-y_k f_b(\mathbf{x}_k))} \sum_{j=1}^n \exp(-y_j f_b(\mathbf{x}_j)) \exp(-\alpha_{b+1} y_j h_{b+1}(\mathbf{x}_j))} \\ &= \frac{\exp(-y_i f_b(\mathbf{x}_i)) \exp(-\alpha_{b+1} y_i h_{b+1}(\mathbf{x}_i))}{\sum_{j=1}^n \exp(-y_j f_b(\mathbf{x}_j)) \exp(-\alpha_{b+1} y_j h_{b+1}(\mathbf{x}_j))} \\ &= \frac{\exp(-y_i [f_b(\mathbf{x}_i) + \alpha_{b+1} h_{b+1}(\mathbf{x}_i)])}{\sum_{j=1}^n \exp(-y_j [f_b(\mathbf{x}_j) + \alpha_{b+1} h_{b+1}(\mathbf{x}_j)])} \end{aligned}$$

Since we by definition have that $f_{b+1}(\mathbf{x}) = f_b(\mathbf{x}) + \alpha_{b+1} h_{b+1}(\mathbf{x})$, the expression reduces to

$$\begin{aligned} w_i^{(b+2)} &= \frac{\exp(-y_i [f_b(\mathbf{x}_i) + \alpha_{b+1} h_{b+1}(\mathbf{x}_i)])}{\sum_{j=1}^n \exp(-y_j [f_b(\mathbf{x}_j) + \alpha_{b+1} h_{b+1}(\mathbf{x}_j)])} \\ &= \frac{\exp(-y_i f_{b+1}(\mathbf{x}_i))}{\sum_{j=1}^n \exp(-y_j f_{b+1}(\mathbf{x}_j))}. \end{aligned}$$

This is exactly $P(b+1)$. This completes the inductive step. So, by induction $P(b)$ holds for all $b \geq 0$, i.e., at each round the weights can be written in the compact form above. $\square$

Response to Question 2. We assume that our weak learner satisfies the guarantee $\varepsilon_b \leq \frac{1}{2} - \gamma$ with probability $1 - \delta'$ for $\delta' \in (0, 1)$, where the probability is over the randomness of selecting the subsampled training set and the randomness of the weak learner.

Now, we have to find $B^* \in \mathbb{Z}_+$ and $\delta^* \in (0, 1)$, such that when AdaBoost runs for B^* iterations with the weak learner, the error of the hypothesis h output by AdaBoost is

$$\frac{1}{n} \sum_{i=1}^n \mathbb{I}(h(x_i) \neq y_i) = 0$$

with probability at least $1 - \delta^*$.

Solution. First, we run AdaBoost for B rounds on the labeled training set T given by

$$T = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_n, y_n)\} \subset \mathbb{R}^d \times \{-1, +1\}$$

using a randomized weak learner, which in each round b returns a hypothesis h_b satisfying

$$\varepsilon_b = \sum_{j=1}^n w_j^{(b)} \mathbb{I}(h_b(\mathbf{x}_j) \neq y_j) \leq \frac{1}{2} - \gamma$$

with probability at least $1 - \delta'$ with randomness over both the sampling of the training subsample D_b and the internal randomness of the weak learner.

The goal is now to find integers $B^* > 0$ and $\delta^* \in (0, 1)$ as functions of (n, γ, δ') , such that if we run the AdaBoost classifier for B^* rounds, then the final hypothesis

$$h(\mathbf{x}) = \text{sign}(f_{B^*}(x)), \quad f_{B^*}(x) = \sum_{b=1}^{B^*} \alpha_b h_b(\mathbf{x})$$

has zero training error

$$\frac{1}{n} \sum_{i=1}^n \mathbb{I}(h(x_i) \neq y_i) = 0$$

with probability at least $1 - \delta^*$. First pretend that every round satisfies $\varepsilon_b \leq \frac{1}{2} - \gamma$. From Theorem 10.2 in Shalev-Shwartz and Ben-David (2014), the training error satisfies

$$\frac{1}{n} \sum_{i=1}^n \mathbb{I}\{h(\mathbf{x}_i) \neq y_i\} \leq \prod_{b=1}^B Z_b \leq \exp(-2\gamma^2 B),$$

where $Z_b = \frac{1}{n} \sum_{i=1}^n \exp(-y_i f_b(\mathbf{x}_i))$. We know make this $\leq$ into a strict $<$ by choosing

$$B^* = \left\lceil \frac{\ln n}{2\gamma^2} + \varepsilon \right\rceil$$

for any $\varepsilon > 0$ to ensure that $B^* \in \mathbb{Z}_+$ and $B^* > \frac{\ln n}{2\gamma^2}$. Furthermore, we have that

$$B^* > \frac{\ln n}{2\gamma^2} \implies -2\gamma^2 B^* < -\ln n \implies \exp(-2\gamma^2 B^*) < \exp(-\ln n) = \frac{1}{n}.$$

Hence, $\frac{1}{n} \sum_{i=1}^n \mathbb{I}\{h(\mathbf{x}_i) \neq y_i\} < \frac{1}{n}$, but the left-hand side is a multiple of $1/n$. The only multiple of $1/n$ strictly below $1/n$ is 0. Therefore, we get that

$$\frac{1}{n} \sum_{i=1}^n \mathbb{I}\{h(\mathbf{x}_i) \neq y_i\} = 0,$$

i.e., the error of the hypothesis h has zero training error. Now, we let $E_b = \{\varepsilon_b \leq \frac{1}{2} - \gamma\}$ be the event "weak learner in round b achieves error $\leq \frac{1}{2} - \gamma$ ". By assumption, it holds that $\Pr[E_b] \geq 1 - \delta'$. We want the probability that all B^* rounds are good, which is

$$\Pr \left[\bigcap_{b=1}^{B^*} E_b \right] = 1 - \Pr \left[\bigcup_{b=1}^{B^*} (E_b)^c \right].$$

By the union bound, we then get that

$$\Pr \left[\bigcup_{b=1}^{B^*} (E_b)^c \right] \leq \sum_{b=1}^{B^*} \Pr[(E_b)^c] = \sum_{b=1}^{B^*} (1 - \Pr[E_b]) \leq \sum_{b=1}^{B^*} \delta' = B^* \delta',$$

such that

$$\Pr \left[\bigcap_{b=1}^{B^*} E_b \right] = 1 - \Pr \left[\bigcup_{b=1}^{B^*} (E_b)^c \right] \geq 1 - B^* \delta'.$$

If we set $\delta^* = B^* \delta'$, then with probability at least $1 - \delta^*$, every round satisfies $\varepsilon_b \leq \frac{1}{2} - \gamma$, so the deterministic bound above applies and drives the training error to zero. To exactly guarantee $\delta^* \in (0, 1)$, we define $\delta^* := \min\{B^* \delta', 1 - \varepsilon\}$ for some arbitrary, small $\varepsilon > 0$.

Thus, with the choice of $B^* = \left\lceil \frac{\ln n}{2\gamma^2} + \varepsilon \right\rceil$ and $\delta^* = \min\{B^* \delta', 1 - \varepsilon\}$ for $\varepsilon > 0$, AdaBoost's final hypothesis has zero training error with probability at least $1 - \delta^*$ as desired.

2 Landcover Classification

In this exercise, we will apply the **AdaBoost** classifier to perform landcover classification using satellite image data from the Sentinel satellites that have a revisit time of 5 days, i.e., they take one image every 5 days for each location that is monitored.

The input consists of two files, **train.npz** and **test.npz**, which contain training and test instances. Each instance is a sequence of 13×13 pixel images from 6 channels across 12 timestamps, along with a class label that indicates the landcover type of the central pixel.

Response to Question 1. Using all pixels as input features can be computationally intensive for **AdaBoost**. To address this, we first reduce the input by selecting only the central pixels from each image as features, resulting in feature vectors with 72 values.

Each instance in our dataset is a small patch of size 13×13 pixels over 6 bands and 12 timestamps with shape `(n_samples, 12, 13, 13, 6)`. Thus, to reduce the input dimensionality, we extract only the central pixel (position `[6, 6]` for a 13×13 image) from each band and timestamp. This gives us 12 (timestamps) $\times$ 6 (bands) = 72 features per instance:

```
# Extract central pixel features: shape (n_samples, 12, 6) -> reshape to (n_samples, 72)
def extract_central_pixels(X):
    # X: (n_samples, 12, 13, 13, 6)
    return X[:, :, 6, 6, :].reshape(X.shape[0], -1)

X_train_central = extract_central_pixels(X_train)
X_test_central = extract_central_pixels(X_test)

print("Reduced training feature shape:", X_train_central.shape)
print("Reduced test feature shape:", X_test_central.shape)

Reduced training feature shape: (6953, 72)
Reduced test feature shape: (1739, 72)
```

Listing 1: Code to extract central pixels from each image as features.

Selecting only the central pixels per image as features is a decent choice for several reasons:

- By reducing the feature space from $13 \times 13 \times 6 \times 12 = 12,168$ features per instance (with all pixels, all bands, all timestamps) to just $6 \times 12 = 72$ features per instance (with only central pixels), it makes each round of the boosted-tree fitting during training much faster and less memory-intensive, which is important for a boosting algorithm like **AdaBoost** that can be slow with high-dimensional data. Given that **AdaBoost** iteratively builds many weak learners on the full feature set, cutting down the feature count by a factor of almost 170 has a notable impact on training time.
- The label for each instance is determined by the class of the central pixel. Therefore, by focusing on the central pixel, the features are directly aligned with the target, which potentially can reduce noise from irrelevant surrounding non-central pixels.

However, this approach also has drawbacks:

- By ignoring surrounding pixels, the model loses information about the spatial structure and context within the image. Neighboring pixels might contain useful clues about landcover type, especially for classes that are hard to distinguish based on a single pixel. For example, if the central pixel happens to be noisy or occluded (e.g. by clouds), we lose the opportunity to infer its true class from neighboring pixels.
- The model may become more sensitive to noise or anomalies in the central pixel, since it cannot "average out" information from the neighborhood.

Response to Question 2. Now, we first have to conduct a 2-fold cross-validation on our training data to select the best-performing **AdaBoost** model. In 2-fold cross-validation, we start by splitting our $N = 6953$ training samples into two roughly equal folds of sizes, where we use the cross-validator `KFold(n_splits=2, shuffle=True, random_state=42)` from **scikit_learn** in Python. We then consider our hyperparameter grid given by

- Depth of decision trees: $d \in \{1, 2, 3\}$
- Number of boosted trees: $m \in \{50, 100, 200\}$

with $3 \times 3 = 9$ settings. For each setting (d, m) , we perform two different AdaBoost trainings (the AdaBoost algorithm is implemented under the “SAMME” multiclass discrete-boosting approach in `scikit-learn` mentioned on slide 32 of Sanyal (2025)), since we train on fold 1 and validate on fold 2, and conversely.

After having performed the grid search, we select the best hyperparameters (d^*, m^*) , then refit one AdaBoost with (d^*, m^*) on all training samples for evaluation on the test set. As weak learners, we consider simple decision-tree classifiers with the Gini index as the impurity measure, done by `DecisionTreeClassifier(max_depth=d, criterion='gini')` for $d = 1, 2, 3$. Below is the code that carries out this procedure using `scikit-learn`:

```
param_grid = {
    'estimator': [DecisionTreeClassifier(max_depth=d, criterion='gini') for d in [1, 2, 3]],
    'n_estimators': [50, 100, 200]
}

class AdaBoostWithDepth(AdaBoostClassifier):
    def set_params(self, **params):
        if 'estimator' in params:
            self.estimator = params['estimator']
            del params['estimator']
        return super().set_params(**params)

cv = KFold(n_splits=2, shuffle=True, random_state=42)

grid = GridSearchCV(
    AdaBoostWithDepth(),
    param_grid,
    cv=cv,
    scoring='accuracy',
    n_jobs=-1,
    refit=True
)
grid.fit(X_train_central, y_train_flat)

print("Best parameters:", grid.best_params_)
print("Best cross-validation accuracy: {:.4f}".format(grid.best_score_))

Best parameters: {'estimator': DecisionTreeClassifier(max_depth=3), 'n_estimators': 200}
Best cross-validation accuracy: 0.7367
```

Listing 2: Code to perform 2-fold cross-validation grid search for AdaBoost with decision trees.

In the code, `param_grid` enumerates all (d, m) combinations by creating one weak learner `DecisionTreeClassifier(max_depth=d, criterion='gini')` for each $d \in \{1, 2, 3\}$ and one $m \in \{50, 100, 200\}$. Because `AdaBoostClassifier` does not allow the `estimator` to be changed on the fly, we subclass it as `AdaBoostWithDepth` and override the `set_params` method so that `GridSearchCV` can replace `estimator` at each grid point. Hereafter, we pass `cv = KFold(n_splits=2, shuffle=True, random_state=42)` into `GridSearchCV`, along with `scoring='accuracy'` and `refit=True`. When `grid.fit(...)` is called, the above procedure to select the best hyperparameters (d^*, m^*) is then done.

From the results, we see that the best model is the one with the hyperparameters

$$(d^*, m^*) = (3, 200)$$

with the best mean cross-validated accuracy given by 0.7367, and by refitting the model with the best hyperparameters (d^*, m^*) to the entire training data with the $N = 6953$ samples, the test accuracy is found to be 0.7188. The code can be found below:

```
best_model = grid.best_estimator_
best_model.fit(X_train_central, y_train_flat)

y_pred = best_model.predict(X_test_central)
test_acc = accuracy_score(y_test_flat, y_pred)
print("Test accuracy: {:.4f}".format(test_acc))

Test accuracy: 0.7188

cm = confusion_matrix(y_test_flat, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[classes[c] for c in
sorted(classes.keys())])
disp.plot(xticks_rotation=45)
plt.title("Confusion Matrix on Test Set")
plt.show()
```

Listing 3: Code to refit the best model, and evaluate on the test set, incl. confusion matrix.

Here, we also show the confusion matrix on the test set to visualize the errors:

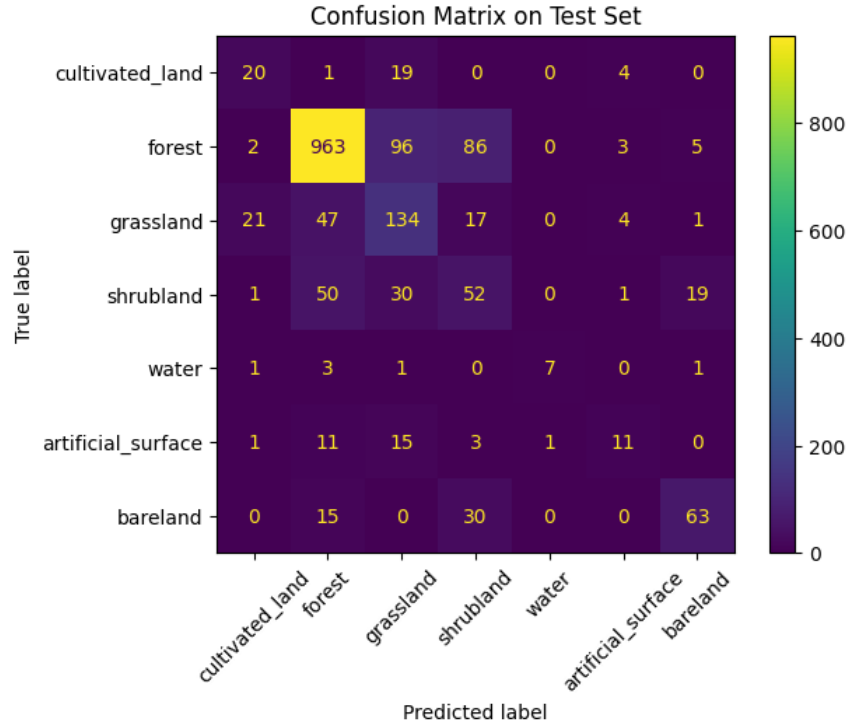


Figure 1: Confusion matrix on the test data (rows = true labels, columns = predicted labels).

It shows that the model performs well on the "forest" class, with most samples correctly classified. However, there is notable confusion between "forest", "grassland", and "shrubland", indicating that the model struggles to distinguish between these vegetation types.

3 Majority Vote

Let MV denote a uniformly weighted majority vote.

Response to Question 1. We have to design an example of $\mathcal{H}$ and decision space $\mathbf{X}$, where $L(\text{MV}) = 0$ and $L(h) \geq \frac{1}{3}$ for all h . First, we let the decision space and label space be $\mathbf{X} = \{x_1, x_2, x_3\}$ and $\mathbf{Y} = \{-1, +1\}$, respectively, where $\mathbf{X} \rightarrow \{-1, +1\}$. We take a uniform distribution p over $\mathbf{X}$, and for the labels $y(x)$ with $p(x) = \frac{1}{3}$, we define

$$y(x_1) = -1, \quad y(x_2) = +1, \quad y(x_3) = +1.$$

Under this uniform p , for any classifier $h: \mathbf{X} \rightarrow \{-1, +1\}$, the expected loss of h is

$$L(h) = \Pr_{x \sim p}[h(x) \neq y(x)] = \frac{|\{x \in \mathbf{X}: h(x) \neq y(x)\}|}{3}.$$

Now, let the $\mathcal{H} = \{h_1, h_2, h_3\}$ be defined by the following mistake patterns:

x	$y(x)$	$h_1(x)$	$h_2(x)$	$h_3(x)$
x_1	-1	-1	-1	+1
x_2	+1	+1	-1	+1
x_3	+1	-1	+1	+1

Table 1: Mistake patterns for $\mathcal{H}$.

Here, we see that h_1 is wrong only at x_3 , so $L(h_1) = \frac{1}{3}$, h_2 is wrong only at x_2 , so $L(h_2) = \frac{1}{3}$, and h_3 is wrong only at x_1 , so $L(h_3) = \frac{1}{3}$. Thus for every hypothesis $h \in \mathcal{H}$, it holds that

$$L(h) = \frac{1}{3} \geq \frac{1}{3}.$$

Since we put a uniform distribution on the three hypotheses, we have that

$$\rho(h_1) = \rho(h_2) = \rho(h_3) = \frac{1}{3}$$

Then, by (3.31) in Seldin (2025), we define the uniformly-weighted majority vote by

$$\text{MV}(\mathbf{X}) = \text{sign} \left(\sum_{h \in \mathcal{H}} \rho(h) h(x) \right) = \text{sign} \left(\frac{1}{3} [h_1(x) + h_2(x) + h_3(x)] \right).$$

For $c = +1$ with $\text{sign}(c) = +1$, it trivially holds that $\text{sign}(cx) = \text{sign}(c)\text{sign}(x) = (+1)\text{sign}(x) = \text{sign}(x)$. Hence, by letting $c = \frac{1}{3} > 0$, we can pull the constant out of the sign operator in the expression for $\text{MV}(\mathbf{X})$ above without changing anything, so that

$$\text{MV}(\mathbf{X}) = \text{sign}(h_1(x) + h_2(x) + h_3(x)).$$

We now check the uniformly-weighted majority vote $\text{MV}(\mathbf{X})$ on each of the three points:

- At x_1 : $h_1(x_1) + h_2(x_1) + h_3(x_1) = (-1) + (-1) + (+1) = -1$, so $\text{MV}(x_1) = \text{sign}(-1) = -1$, which matches the true label $y(x_1) = -1$.
- At x_2 : $h_1(x_2) + h_2(x_2) + h_3(x_2) = (+1) + (-1) + (+1) = +1$, so $\text{MV}(x_2) = \text{sign}(+1) = +1$, which matches the true label $y(x_2) = +1$.
- At x_3 : $h_1(x_3) + h_2(x_3) + h_3(x_3) = (-1) + (+1) + (+1) = +1$, so $\text{MV}(x_3) = \text{sign}(+1) = +1$, which matches the true label $y(x_3) = +1$.

In all three cases, the weighted majority vote $\text{MV}(\mathbf{X})$ is equal to the true label $y(\mathbf{X})$, i.e., $\text{MV}(\mathbf{X}) = y(\mathbf{X})$. Thus, $\text{MV}(\mathbf{X})$ never errs on any of the three points and is always correct,

$$L(\text{MV}) = \Pr_{x \sim p}[\text{MV}(x) \neq y(x)] = \frac{0}{3} = 0.$$

Hence, we have designed an example of $\mathcal{H}$ and $\mathbf{X}$ with $|\mathcal{H}| = 3$ and $|\mathbf{X}| = 3$, such that the hypotheses satisfy $L(h_i) = \frac{1}{3} \geq \frac{1}{3}$ for $i = 1, 2, 3$, while majority vote satisfies $L(\text{MV}) = 0$.

Response to Question 2. We have to design an example of $\mathcal{H}$ and $\mathbf{X}$, where $L(\text{MV}) > L(h)$ for all $h \in \mathcal{H}$. As in the previous question, we first let the decision space and label space be $\mathbf{X} = \{x_1, x_2, x_3\}$ and $\mathbf{Y} = \{-1, +1\}$, respectively, where $\mathbf{X} \rightarrow \{-1, +1\}$ with the uniform distribution $p(x_i) = \frac{1}{3}$ for each $i = 1, 2, 3$. For the labels $y(x)$, we now define

$$y(x_1) = -1, \quad y(x_2) = -1, \quad y(x_3) = -1.$$

Under this uniform distribution p , a hypothesis $h: \mathbf{X} \rightarrow \{-1, +1\}$ has

$$L(h) = \Pr_{x \sim p}[h(x) \neq y(x)] = \frac{|\{x \in \mathbf{X}: h(x) \neq y(x)\}|}{3}.$$

Since $y(x_i) = -1$ for all $i = 1, 2, 3$, an error on x_i means exactly $h(x_i) = +1$. Now, let $\mathcal{H} = \{h_1, h_2, h_3\}$. In our example, we want each h_i to output $+1$ (making an error) on exactly two of the three points, and output -1 (correct) on the last one. Also, at each fixed x_i , we want exactly two of the three hypotheses to output $+1$. That forces the majority vote to be $+1$ (and hence wrong) on every x_i . $\mathcal{H}$ has the following mistake patterns:

x	$y(x)$	$h_1(x)$	$h_2(x)$	$h_3(x)$
x_1	-1	$+1$	-1	$+1$
x_2	-1	$+1$	$+1$	-1
x_3	-1	-1	$+1$	$+1$

Table 2: Mistake patterns for $\mathcal{H}$.

Here, each hypothesis $h_i \in \mathcal{H}$ for $i = 1, 2, 3$ makes two mistakes, such that

$$L(h_i) = \frac{2}{3}.$$

Since we still put a uniform distribution on the three hypotheses, we have that

$$\rho(h_1) = \rho(h_2) = \rho(h_3) = \frac{1}{3}$$

Using (3.31) in Seldin (2025), and observing that multiplying by the positive constant $\frac{1}{3} > 0$ does not change the sign as in Question 1, the uniformly weighted majority vote is

$$\begin{aligned} \text{MV}(\mathbf{X}) &= \text{sign} \left(\sum_{h \in \mathcal{H}} \rho(h) h(x) \right) \\ &= \text{sign} \left(\frac{1}{3} [h_1(x) + h_2(x) + h_3(x)] \right) \\ &= \text{sign}(h_1(x) + h_2(x) + h_3(x)) \end{aligned}$$

As before, we check the uniformly-weighted majority vote $\text{MV}(\mathbf{X})$ on each point:

- At x_1 : $h_1(x_1) + h_2(x_1) + h_3(x_1) = (+1) + (-1) + (+1) = +1$, so $\text{MV}(x_1) = \text{sign}(+1) = +1$, which does not match the true label $y(x_1) = -1$, and the majority vote errs.
- At x_2 : $h_1(x_2) + h_2(x_2) + h_3(x_2) = (+1) + (+1) + (-1) = +1$, so $\text{MV}(x_2) = \text{sign}(+1) = +1$, which does not match the true label $y(x_2) = -1$, and the majority vote errs.
- At x_3 : $h_1(x_3) + h_2(x_3) + h_3(x_3) = (-1) + (+1) + (+1) = +1$, so $\text{MV}(x_3) = \text{sign}(+1) = +1$, which does not match the true label $y(x_3) = -1$, and the majority vote errs.

Thus, MV is wrong on all three points $x_i \in \mathbf{X}$ for $i = 1, 2, 3$, so

$$L(\text{MV}) = \Pr_{x \sim p} [\text{MV}(x) \neq y(x)] = \frac{3}{3} = 1,$$

which for all hypothesis $h_i \in \mathcal{H}$ for $i = 1, 2, 3$ as desired means that

$$L(\text{MV}) = 1 > \frac{2}{3} = L(h_i), \quad i = 1, 2, 3.$$

Response to Question 3. We let $\mathcal{H}$ be a hypothesis space, such that $|\mathcal{H}| = M$ and all $h \in \mathcal{H}$ have the same expected error $L(h) = \frac{1}{2} - \varepsilon$ for $\varepsilon > 0$, and that the hypotheses in $\mathcal{H}$ make independent errors. Now, we have to prove that $L(\text{MV}) \xrightarrow{|\mathcal{H}| \rightarrow \infty} 0$.

Proof. First, we let $\mathcal{H} = \{h_1, h_2, \dots, h_M\}$ be a collection of M classifiers (hypotheses) mapping $\mathbf{X} \rightarrow \{-1, +1\}$. Then, we suppose that, for every classifier $h \in \mathcal{H}$, it holds that

$$L(h) = \Pr_{(x,y) \sim \mathcal{D}} [h(x) \neq y(x)] = \frac{1}{2} - \varepsilon$$

for a fixed constant $\varepsilon > 0$. Assume that for a random draw $(x, y) \sim \mathcal{D}$, the error-events

$$E_i := \{h_i(x) \neq y(x)\}$$

are independent across $i = 1, 2, \dots, M$. We wish to show that the majority-vote classifier

$$\text{MV}(\mathbf{X}) = \text{sign} \left(\sum_{i=1}^M \rho(h_i) h_i(x) \right) = \text{sign} \left(\sum_{i=1}^M h_i(x) \right),$$

since $\rho(h) = \frac{1}{M}$ are positive and does not change the sign, with $|\mathcal{H}| = M$ has error

$$L(\text{MV}) = \Pr_{(x,y) \sim \mathcal{D}} [\text{MV}(x) \neq y(x)] \xrightarrow{M \rightarrow \infty} 0.$$

For a fixed draw $(x, y) \sim \mathcal{D}$, we define

$$Z_i := \mathbf{1}\{h_i(x) \neq y(x)\} = \begin{cases} 1, & \text{if } h_i(x) \neq y(x) \\ 0, & \text{if } h_i(x) = y(x). \end{cases}$$

By assumption, we have that

$$\Pr[Z_i = 1] = \Pr[h_i(x) \neq y(x)] = \frac{1}{2} - \varepsilon, \quad \Pr[Z_i = 0] = 1 - \left(\frac{1}{2} - \varepsilon\right) = \frac{1}{2} + \varepsilon.$$

The $Z_1, \dots, Z_M$ are independent Bernoulli-random variables (for each fixed (x, y)). Since,

$$\text{MV}(x) \neq y(x) \Leftrightarrow \text{more than half of the } h_i \text{ are wrong on } (x, y) \Leftrightarrow \sum_{i=1}^M Z_i \geq \frac{M}{2},$$

we can equivalently set

$$\bar{Z}_M := \frac{1}{M} \sum_{i=1}^M Z_i,$$

which means that $\{\text{MV}(x) \neq y(x)\} \subseteq \{\bar{Z}_M \geq \frac{1}{2}\}$. Therefore, we now see that

$$L(\text{MV}) = \Pr_{(x,y) \sim \mathcal{D}} [\text{MV}(x) \neq y(x)] = \Pr \left[\bar{Z}_M \geq \frac{1}{2} \right].$$

Since each Z_i is Bernoulli($p = \frac{1}{2} - \varepsilon$) and they are independent, it holds that

$$\mathbb{E}[Z_i] = \frac{1}{2} - \varepsilon, \quad \mathbb{E}[\bar{Z}_M] = \frac{1}{M} \sum_{i=1}^M \mathbb{E}[Z_i] = \frac{1}{M} \cdot M \left(\frac{1}{2} - \varepsilon \right) = \frac{1}{2} - \varepsilon.$$

Denote this common mean by $\mu := \mathbb{E}[\bar{Z}_M] = \frac{1}{2} - \varepsilon$. Hence, we can rewrite

$$\Pr \left[\bar{Z}_M \geq \frac{1}{2} \right] = \Pr \left[\bar{Z}_M - \mu \geq \frac{1}{2} - \left(\frac{1}{2} - \varepsilon \right) \right] = \Pr [\bar{Z}_M - \mu \geq \varepsilon].$$

Each $Z_i \in \{0, 1\}$, so $Z_i \in [0, 1]$. Because the Z_i are also independent, Hoeffding's inequality from Corollary 2.5 in Seldin (2025) gives us that

$$\Pr [\bar{Z}_M - \mu \geq \varepsilon] \leq \exp(-2M\varepsilon^2),$$

such that the uniformly weighted majority votes is bounded by

$$L(\text{MV}) = \Pr \left[\bar{Z}_M \geq \frac{1}{2} \right] = \Pr [\bar{Z}_M - \mu \geq \varepsilon] \leq \exp(-2M\varepsilon^2)$$

Because the bound $\exp(-2M\varepsilon^2)$ does not depend on the particular (x, y) drawn from $\mathcal{D}$, it holds uniformly over all (x, y) . Hence, we get

$$L(\text{MV}) = \Pr_{(x,y) \sim \mathcal{D}} [\text{MV}(x) \neq y(x)] \leq \exp(-2M\varepsilon^2).$$

As $M \rightarrow \infty$, the right-hand side $\exp(-2M\varepsilon^2) \rightarrow 0$. In other words, with $|\mathcal{H}| = M$, we get

$$L(\text{MV}) \xrightarrow{|\mathcal{H}| \rightarrow \infty} 0,$$

i.e., $L(\text{MV})$ converges to zero exponentially fast with the growth of M as claimed. $\square$

4 Occam's kl-razor vs. PAC-Bayes-kl

Response to Question 1. We have to prove the following theorem, which extends Theorem 3.38 from Exercise 3.1 in Seldin (2025) to soft selection:

Theorem 3.40 (Occam's kl-razor inequality for soft selection). *Under the conditions of Theorem 3.38, it holds that*

$$\mathbb{P} \left(\exists \rho: \text{kl} \left(\mathbb{E}_\rho[\hat{L}(h, S)] \| \mathbb{E}_\rho[L(h)] \right) \geq \frac{\mathbb{E}_\rho \left[\ln \frac{1}{\pi(h)} \right] + \ln \frac{1}{\delta}}{n} \right) \leq \delta$$

Proof. First, we recall Theorem 3.38 with hard-selection Occam's kl-razor given by

$$\mathbb{P} \left(\exists h \in \mathcal{H}: \text{kl}(\hat{L}(h, S) \| L(h)) \geq \frac{\ln \frac{1}{\pi(h)\delta}}{n} \right) \leq \delta.$$

Equivalently, with probability at least $1 - \delta$ (over the draw of S), every $h \in \mathcal{H}$ satisfies

$$\text{kl}(\hat{L}(h, S) \| L(h)) \leq \frac{\ln \frac{1}{\pi(h)\delta}}{n} \Leftrightarrow \text{kl}(\hat{L}(h, S) \| L(h)) \leq \frac{\ln \left(\frac{1}{\pi(h)} \right) + \ln \left(\frac{1}{\delta} \right)}{n}. \quad (\star)$$

Then, we fix ρ to be any posterior probability distribution over $\mathcal{H}$. Since $\text{kl}(p\|q)$ is jointly convex in $(p, q) \in [0, 1] \times [0, 1]$, Jensen's inequality for convex functions gives us that

$$\text{kl}\left(\mathbb{E}_{h \sim \rho}[\hat{L}(h, S)] \parallel \mathbb{E}_{h \sim \rho}[L(h)]\right) \leq \mathbb{E}_{h \sim \rho} \left[\text{kl}(\hat{L}(h, S) \parallel L(h)) \right]. \quad (\star\star)$$

Because inequality $(\star)$ is simultaneously valid for every $h \in \mathcal{H}$ with probability at least $1 - \delta$, we can take the expectation with respect to any distribution ρ , such that

$$\mathbb{E}_{h \sim \rho}[\text{kl}(\hat{L}(h, S) \parallel L(h))] \leq \mathbb{E}_{h \sim \rho} \left[\frac{\ln\left(\frac{1}{\pi(h)}\right) + \ln\left(\frac{1}{\delta}\right)}{n} \right] = \frac{\mathbb{E}_{h \sim \rho} \left[\ln \frac{1}{\pi(h)} \right] + \ln \frac{1}{\delta}}{n},$$

where we could take $\ln \frac{1}{\delta}$ and $\frac{1}{n}$ out of the expectation, since they are constants. By combining $(\star\star)$ with the above expression, we immediately deduce that with probability at least $1 - \delta$, it holds for every posterior ρ that

$$\text{kl}\left(\mathbb{E}_{h \sim \rho}[\hat{L}(h, S)] \parallel \mathbb{E}_{h \sim \rho}[L(h)]\right) \leq \frac{\mathbb{E}_{h \sim \rho} \left[\ln \frac{1}{\pi(h)} \right] + \ln \frac{1}{\delta}}{n}.$$

Since this statement is valid simultaneously for all choices of ρ , it follows that

$$\mathbb{P} \left(\exists \rho: \text{kl}\left(\mathbb{E}_{h \sim \rho}[\hat{L}(h, S)] \parallel \mathbb{E}_{h \sim \rho}[L(h)]\right) \geq \frac{\mathbb{E}_{h \sim \rho} \left[\ln \frac{1}{\pi(h)} \right] + \ln \frac{1}{\delta}}{n} \right) \leq \delta.$$

But this is exactly the statement of Theorem 3.40. Therefore, the proof is complete. $\square$

Response to Question 2. We have to compare Theorem 3.40 to PAC-Bayes-kl inequality in Theorem 3.26, which is given by

Theorem 3.26 (PAC-Bayes-kl inequality). *For any “prior” distribution π over $\mathcal{H}$ that is independent of S and $\delta \in (0, 1]$,*

$$\mathbb{P} \left(\exists \rho: \text{kl}\left(\mathbb{E}_{h \sim \rho}[\hat{L}(h, S)] \parallel \mathbb{E}_{h \sim \rho}[L(h)]\right) \geq \frac{\text{KL}(\rho \parallel \pi) + \ln \frac{2\sqrt{n}}{\delta}}{n} \right) \leq \delta$$

and assess the advantages and disadvantages of PAC-Bayes-kl.

First, we see that the left-hand sides of both bounds quantify exactly the same event:

$$\exists \rho: \text{kl}\left(\mathbb{E}_{h \sim \rho}[\hat{L}(h, S)] \parallel \mathbb{E}_{h \sim \rho}[L(h)]\right) \geq \dots,$$

so both theorems control “how far the empirical loss can deviate from the true loss for any posterior distribution ρ . The only difference lies in the right-hand-side penalty, where theorem 3.40 with the soft Occam's kl-razor charges

$$\frac{\mathbb{E}_{h \sim \rho} \left[\ln \frac{1}{\pi(h)} \right] + \ln \frac{1}{\delta}}{n},$$

Thus, you pay the prior-average of $\ln(1/\pi(h))$ plus $\ln(1/\delta)$, all divided by n . There is no dependence on the entropy of ρ . In contrast, the PAC-Bayes-kl inequality charges

$$\frac{\text{KL}(\rho\|\pi) + \ln \frac{2\sqrt{n}}{\delta}}{n},$$

where we from page 46 in Seldin (2025) know that

$$\text{KL}(\rho\|\pi) = \mathbb{E}_\rho \left[\ln \frac{\rho(h)}{\pi(h)} \right] = \underbrace{\mathbb{E}_\rho \left[\ln \frac{1}{\pi(h)} \right]}_{\text{Average complexity}} - \underbrace{H(\rho)}_{\text{Entropy}}$$

with the entropy of the posterior distribution ρ given by $H(\rho) = -\sum_{h \in \mathcal{H}} \rho(h) \ln \rho(h)$ by Definition 2.9 in Seldin (2025), such that the PAC-Bayes-kl inequality now charges

$$\frac{\mathbb{E}_\rho \left[\ln \frac{1}{\pi(h)} \right] - H(\rho) + \ln \frac{2\sqrt{n}}{\delta}}{n}.$$

So, PAC-Bayes subtracts $H(\rho)$ from $\mathbb{E}_\rho[\ln(1/\pi(h))]$ unlike the soft bound. We can see $-H(\rho)$ as a “discount” when ρ spreads its probability mass over multiple prediction rules (larger entropy). The soft bound pays $\ln(1/\delta)/n$, and PAC-Bayes pays $\ln(2\sqrt{n}/\delta)/n$, where

$$\ln \frac{2\sqrt{n}}{\delta} = \ln \frac{1}{\delta} + \ln(2\sqrt{n}).$$

Hence, PAC-Bayes introduces a small extra $\ln(2\sqrt{n})/n$ “overhead” that does not appear in the Soft Occam’s bound. If H is countable and ρ assigns all its probability mass to a single prediction rule (i.e., a hard selection), then $H(\rho) = 0$, and there is no “bonus” term. Then, both bounds charge $\mathbb{E}_\rho[\ln(1/\pi(h))]/n$ plus “small” terms as mentioned above.

Thus, the advantages of PAC-Bayes-kl include its “entropy discount”. Because it charges $\mathbb{E}_\rho[\ln 1/\pi(h)] - H(\rho)$, any spread in ρ (larger $H(\rho)$) reduces the penalty. When n grows, the $\ln(2\sqrt{n})/n$ overhead becomes negligible, so whenever one can choose a high-entropy ρ that averages over several good predictors, it yields a smaller term and hence tighter bound.

But, the disadvantages of PAC-Bayes-kl are that it introduces a $\ln(2\sqrt{n})/n$ overhead absent from the soft bound, which can dominate when n is small. It also requires computing or approximating $\text{KL}(\rho\|\pi)$ that can be cumbersome if $\mathcal{H}$ is large or continuous. And, when ρ collapses to a single hypothesis ($H(\rho) = 0$) PAC-Bayes’ only benefit disappears and its extra $\ln(2\sqrt{n})/n$ term makes it slightly looser than the soft Occam’s penalty.

References

Sanyal, A. (2025). Slides: Boosting I (AdaBoost).

Seldin, Y. (2025). Machine Learning: The Science of Selection under Uncertainty.

Shalev-Shwartz, S. and Ben-David, S. (2014). Understanding Machine Learning: From Theory to Algorithms.